# Exact Integer Alias Method: Robust and Error-Free Large-Scale Sampling

Oriane Crouzet[1], Simon Dreyer[2], Antoine Genitrini[1], and Mehdi Naima[1]

[1] Sorbonne Université, CNRS, LIP6, F-75005 Paris, France
Oriane.Crouzet@lip6.fr, Antoine.Genitrini@lip6.fr, Mehdi.Naima@lip6.fr
[2] Université de Montpellier, CNRS, LIRMM, F-34095 Montpellier, France
Simon.Dreyer@lirmm.fr

**Abstract.** Walker's Alias Method, made linear by Vose, is the gold standard for sampling from a fixed discrete distribution in constant time, but its reliance on floating-point arithmetic makes it unreliable once weights exceed double-precision resolution, a regime already reached by applications such as combinatorial chemistry. Existing remedies either fall back on costly arbitrary-precision arithmetic, like the Rust `weighted_rand` crate. We introduce the Integer-Alias Sampler, a construction and sampling scheme that operates natively and exclusively on arbitrary-precision integer weights. We prove its correctness, give tight bounds on the resulting table size, and analyze the random-bit consumption of the associated generator. Experiments, including a large-scale combinatorial-chemistry case study, show our sampler achieves the best available trade-off between sampling time, memory footprint, and randomness consumption once weights genuinely require arbitrary precision.

**Keywords:** Alias method · Exact random sampling · Arbitrary-precision arithmetic · Discrete random variate generation.

## 1 Introduction

Sampling a discrete object from a prescribed, possibly non-uniform, distribution is a recurring primitive in computational science, underlying Monte Carlo simulation, agent-based modeling, and the stochastic components of modern machine learning pipelines, where billions of independent draws must be produced under strict time and memory budgets. The related question of generating or sampling from a large, implicitly described universe of discrete structures has likewise been studied extensively, from the classical treatises of Nijenhuis and Wilf [9] and Kreher and Stinson [4] to Ruskey's survey [10], more recent developments [2], and Skiena's algorithmic toolbox [13]. In practice, the objects to be generated are rarely equally likely: they carry integer or rational *weights*, and an unbiased sampler must reproduce the corresponding distribution exactly, however skewed it is.

The reference solution to this weighted sampling problem is Walker's *Alias Method* [17,18]: it reorganizes a set of $n$ unequal probabilities into $n$ buckets

of equal capacity, each holding at most two outcomes, a primary value and its designated *alias*, so that a single bucket, drawn uniformly at random, together with one biased coin flip, suffices to recover the target distribution. Vose later showed how to build this table in linear time [16], and Smith and Jacobson provided a thorough analysis of the resulting sampler [14]. By front-loading the whole complexity of the distribution into a one-time linear-time preprocessing step, the Alias Method achieves $O(1)$ sampling independently of $n$, making it the gold standard whenever a fixed distribution must be queried a very large number of times. This is the setting of this paper: the distribution never changes, and every sampler we consider works in two phases, a preprocessing step performed once, then a sampling step repeated as many times as needed, each with its own time and memory cost.

This efficiency, however, is traditionally obtained at the price of floating-point arithmetic, and modern computational-science workloads increasingly expose the fragility of this choice. Both the construction of the table and the sampling step rely on repeatedly comparing and renormalizing probabilities represented as IEEE 754 doubles, so that rounding drift can silently corrupt the classification of outcomes during preprocessing, and bit-creep can corrupt the coin-flip threshold during sampling. As soon as the total weight of a distribution exceeds $2^{53}$, the granularity of double-precision floats becomes coarser than the weight of individual items, which then acquire a *structurally zero* probability of ever being drawn. Such regimes are no longer isolated cases: in combinatorial chemistry, the space of chemically plausible molecules grows so quickly with the number of atoms that it must be sampled rather than enumerated, with reference libraries already ranging from about $10^8$ molecules for the smallest structures to several trillion compounds for modern make-on-demand libraries [15], well beyond $2^{64}$ for molecules of about thirty heavy atoms. Monterrubio-Chanca and von Rudorff [7], whose open-source `nablachem` package [8] we use in our own experiments, address this regime with a pipeline whose final step is precisely a weighted random draw among combinatorial groups, over weights of several hundred bits, well beyond the reach of a double-precision mantissa.

Several directions have been explored to make the Alias Method robust to this kind of scale. The most direct remedy is to abandon floating-point numbers altogether and represent probabilities as arbitrary-precision rational fractions, which restores mathematical exactness but incurs a substantial overhead in both running time and memory, since every comparison and table entry now manipulates a pair of big integers. An alternative, adopted by the Rust ecosystem's `weighted_rand` crate [11], is to restrict the input to non-negative integer weights, so that the whole construction can be carried out with exact integer arithmetic; this avoids fractions entirely, but since its construction departs from Vose's, its correctness does not follow from Vose's proof and had, to the best of our knowledge, never been established in the literature, and, as we show in Section 3, it also produces alias tables with systematically fewer alias-free cells than Vose's construction, which slows down the sampling step. Orthogonally, entropy-optimal samplers such as FLDR [12] and its refinement ALDR [1] build

on the discrete-distribution-generating trees of Knuth and Yao [3] to approach the information-theoretic minimum number of random bits per sample, at the cost of a preprocessing and sampling cost that grows with the entropy of the distribution, again without native support for arbitrary-precision weights.

In this paper we introduce the *Integer-Alias Sampler*, a construction and sampling scheme that operates natively and exclusively on arbitrary-precision integer weights, without ever resorting to floating-point numbers or rational fractions. We revisit Walker and Vose's construction in this exact integer setting, formally establish the correctness of the Rust integer-weight variant [11], and prove that our own construction, built around a natural integer cell size, yields alias tables with more alias-free cells, and hence a faster sampling step, together with matching tight bounds on the size of the resulting table (Sections 2 and 3). We design an efficient random generation procedure on this table, together with a virtual-object rejection mechanism, and analyze its random-bit consumption against the information-theoretic optimum realized by Lumbroso's algorithms [6] (Section 4). We implement all of the above under arbitrary-precision arithmetic, coupled with a background thread that continuously refills a cryptographically secure random-bit stream, overlapping bit generation with sampling, supporting distributions whose number of objects and individual weights can both grow without any fixed-width bound. Finally, we experimentally compare our approach against the Fraction-Alias sampler, the Rust-Alias sampler [11], and arbitrary-precision adaptations of FLDR [12] and ALDR [1], including a large-scale combinatorial-chemistry case study derived from [7,15], showing that our sampler achieves the best available trade-off between sampling time, memory footprint, and random-bit consumption once weights genuinely require arbitrary precision (Section 5).

## 2 Alias algorithm beyond floating numbers rounding errors

The *Alias Method*, designed by Walker in consecutive papers [17,18], represents a fundamental shift in how we sample from discrete probability distributions. His somehow geometric approach transforms a set of $n$ unequal probabilities into $n$ uniform buckets of equal capacity. By reallocating the surplus from high-probability outcomes to fill the deficits of low-probability ones, Walker demonstrated that any distribution can be restructured so that each bucket contains at most two outcomes: a primary value and its designated *alias*. This structure allows for an exceptionally fast sampling process operating in constant time, regardless of the total number of possible outcomes, as it only requires picking a bucket uniformly at random and performing a single biased coin flip to determine the result (i.e. which of the two elements from the bucket to return).

While Walker's sampling approach is extremely smart, the algorithm's modern efficiency is largely attributed to Vose's 1991 refinement, which introduced a linear-time construction method for the bucket table. This phase will be called *preprocessing* in what follows. In his paper [16], Vose optimized this process to

linear time in the number of elements by utilizing two worklists, (implemented as stacks for example), to classify the outcomes as either small or large relative to the mean. The systematic pairing of underfilled and overfilled buckets ensures a stable and computationally lean initialization, improving the super linear time complexity preprocessing proposed by Walker. Consequently, Vose's approach transformed the Alias Method into a highly scalable tool, making it the definitive choice for high-performance computing and large-scale simulations.

The synergy between Walker's conceptual framework and Vose's preprocessing optimization makes the Alias Method the gold standard for scenarios requiring massive numbers of samples from a fixed distribution. Its ability to maintain the same execution speed whether dealing with ten or ten million objects gives it a distinct advantage over traditional binary search methods, which inevitably slow down as the dataset grows.

The practical implementation of the Alias Method must contend with the realities of finite-precision floating-point arithmetic, which introduces subtle vulnerabilities during both the construction and simulation phases. During table initialization, the iterative process of normalizing probabilities and reallocating surpluses can lead to precision drift, where cumulative rounding errors cause the total probability mass to deviate from unity or result in the misclassification of outcomes into the worklists. These inaccuracies persist into the simulation phase, where the sampling step relies on comparing a generated random variable (with a given precision) against a threshold that may have been slightly corrupted by bit-creep. Even a microscopic deviation (such as a probability intended to be exactly 1.0 being stored as an infinitesimally smaller fraction) can erroneously trigger the alias outcome, while the discrete granularity of the underlying pseudo-random number generator can further distort the intended statistical distribution over the course of massive Monte Carlo trials.

To address rounding errors in both the preprocessing and simulation phases, the most effective approach is to replace standard floating-point arithmetic with arbitrary-precision fractions. In practice, since the floating number have finite precision, the probabilities can be represented as rational numbers, i.e. as fractions of arbitrary size. This technique effectively treats the input as a mathematically perfect probability distribution (without any consideration on approximations in the inputs), eliminating the precision drift and bit-creep that typically plague massive Monte Carlo trials. By maintaining exact rational representations, the software ensures that the statistical integrity of the simulation remains uncompromised, regardless of the number of iterations performed. An adaptation of Vose's preprocessing is presented in Algorithm 1 through the function Table_Fractions. Due to the fact that no rounding error occur, the code is simpler than Vose's original implementation. However, relying on exact rational arithmetic introduces significant computational overhead and increased memory usage compared to native floating-point types, which impact overall performance in large-scale simulations.

To improve this aspect, developers for the Rust programming language proposed an alternative approach [11], relying on a distribution of the objects

based on some non-negative integer weights. This restriction is directly reflected in the probability distribution of the objets, because the probability of any given object is then a rational number. Rust's adaptation consider a sequence $S = [x_0, x_1, \dots x_{n-1}]$ of $n$ distinct objects and a sequence $W = [w_0, \dots, w_{n-1}]$ of non-negative integers such that object $x_i$ is of weight $w_i$. We denote the total weight by $w = \sum_i w_i$.

We aim at generating at random an object $x_i$ according to the global distribution $W$, i.e. the probability for generating $x_i$ is given by $w_i/w$, for each $i$. Obviously the later distribution is a probability distribution. Say differently, a sequence of objects with rational probabilities can be equivalently represented with non-negative integer weights replacing each probability $p_i$ by the integer $w_i = p_i \cdot LCM$ where $LCM$ is the least common multiple of the denominators of the probabilities $(p_i)_i$.

Only the preprocessing phase of the Rust implementation is distinct from the original Alias method, the sampling phase remains identical. We put together both approaches in Algorithm 1 for a better comparison. Note that the Rust implementation takes advantage of polymorphism, hence the same implementation should apply either for floating point numbers, but also for integer weights distributions. So in the following we simplify a little the pseudo-code for focusing only on necessary parts used for an integer distribution weights for objects.

The inputs are $X$ and $W$; we deduce $n$ the number of objects of $X$ and $w$ the cumulative weight of all objects. We aim at producing a table $T$ of $n$ buckets, each one with the same capacity (1 in Vose's case and $w$ for Rust). Each bucket is a triplet of the form $(w', x', x)$ where, for Rust, $w'$ is the weight of the object $x'$ in the bucket and $x$ is the alias of $x'$ (implicitly, the weight of $x$ in the bucket is $w - w'$). For Vose, $w'$ is the probability for choosing $x'$ in the bucket , and this $1 - w'$ the one for $x$, the alias.

Vose established the algorithm's correctness for exact arithmetic by proving that $S$ cannot be exhausted before $L$. This ensures that any objects remaining at line 18 have a weight of exactly 1 and require no alias. Rust's implementation differs: after scaling by $n$, an object whose weight equals the cell capacity $w$ (that is, an object of exactly average weight $w/n$) is classified as large (put in $L$). Intuitively, this change should give more possibilities to complete smaller objects, thus the correctness should not be affected. But to the best of our knowledge, this approach provided by Rust has not been proved correct in the literature. Furthermore, in the implementation the loop condition (line 8) is different `while S and L are not empty`, and also there is third loop, line 22 to empty the stack $S$. We will prove it is non necessary in the context of integer weights distribution.

**Proposition 1.** *Rust algorithm is correct for the building of the Alias Table.*

The proof key-ideas are similar than those we present for our algorithm in Theorem 1.

Both tables produced by Vose and Rust have the same number of buckets, but the number of buckets without aliases may differ.

**Algorithm 1** Building of the Alias Tables by Vose (adaptation for fractions) and by Rust language

**Input:** $X$, $W$, $n$ previously defined.
**Output:** The table of triplets $T$.

```
1  function TABLE_FRACTIONS(X, n, W)
2      T = init(n)
3      for x ∈ X do
4          W[x] = n · W[x]
5      L = {x ∈ X | W[x] > 1}
6      S = {x ∈ X | W[x] ≤ 1}
7      k = 0
8      while L is not empty do
9          x = pop(L)
10         x' = pop(S)
11         T[k] = (W[x'], x', x)
12         W[x] = W[x] − (1 − W[x'])
13         if W[x] ≤ 1 then
14             push(S, x)
15         else
16             push(L, x)
17         k = k + 1
18     while S is not empty do
19         x = pop(S)
20         T[k] = (1, x, ∅)
21         k = k + 1
22     return T
```

**Input:** $X$, $W$, $n$ and $w$ previously defined.
**Output:** The table of triplets $T$.

```
1  function TABLE_RUST(X, n, W, w)
2      T = init(n)
3      for x ∈ X do
4          W[x] = n · W[x]
5      L = {x ∈ X | W[x] ≥ w}
6      S = {x ∈ X | W[x] < w}
7      k = 0
8      while S is not empty do
9          x = pop(L)
10         x' = pop(S)
11         T[k] = (W[x'], x', x)
12         W[x] = W[x] − (w − W[x'])
13         if W[x] < w then
14             push(S, x)
15         else
16             push(L, x)
17         k = k + 1
18     while L is not empty do
19         x = pop(L)
20         T[k] = (w, x, ∅)
21         k = k + 1
22     return T
```

- `init`$(n)$: Initialization of a table indexed from 0 to $n-1$
- $L$ is the stack of the larger objects; $S$ is the stack of the smaller objects
- `pop`$(\cdot)$: pop operation extracts one object from the stack
- `push`$(\cdot,\cdot)$: push operation adds one object to the stack

**Proposition 2.** *The number of cells without aliases in the table produced by Rust is smaller than or equal to the one in the table by Vose. Hence, for the sampling step, the approach by Rust is less efficient than the one of Vose sampling.*

In fact, for the sampling step, after having chosen uniformly a bucket, one needs to determine which object among the main object or its alias to return. If the bucket has no alias, then the sampling is done in constant time. Otherwise, one needs to perform a biased coin flip to determine which object to return. Hence, the more buckets without aliases, the more efficient the sampling step.

Let us provide an extreme case to compare Vose's and Rust's approaches. Consider a set of $n$ objects where all objects have weight 2, except for the first two objects which have respectively weights 1 and 3. The total weight is $w = 2n$, i.e., the average weight is 2. In Vose's approach, the first $n-2$ objects (all with weight 2) are directly placed into the result table without aliases. In Rust's approach, due to the LIFO policy (Last-In, First-Out for stacks $L$ and $S$, but $X$ is a table enumerated from index 0 to $n-1$), and the classification of objects of average weight as large, the processing order is different. The largest object (weight 3) is processed after other large objects. Each object with weight 2 is used to complete a small object. This lead to a situation where only a single object ends up in the return table without an alias, while all other cells are divided into two cells.

*Proof.* During the initialization of stacks $L$ and $S$, objects are added in the same order as they appear in the initial set $X$. However, a crucial difference arises from the Rust implementation's use of LIFO stacks for $L$ and $S$. This means objects are processed in the reverse order of their initial placement.

In Vose's approach, objects with a weight exactly equal to $w$ are classified as small and are directly placed into the result table $T$ without aliases. They perfectly fill a bucket, thus requiring no secondary alias.

In Rust's version, the only objects that are guaranteed to have no aliases in the final table are those that remain in $L$ after all objects from $S$ have been processed. These would typically be the first objects from $X$ that were pushed onto $L$ with a weight of exactly $w$, or for at most one object, its remaining portion (it was initially heavier than $w$) that has been reduced to weight $w$. If this specific case occurs in Rust's approach, it is also the case in Vose's.

In summary, Vose's approach ensures that all elements whose weight is $w$ are directly placed without aliases, leading to a higher number of alias-free entries in the resulting table. This is generally not the case for Rust's version, which can introduce aliases for objects that initially had the ideal weight $w$, making Vose's approach more efficient in terms of alias reduction for the sampling step. □

In conclusion, while Rust approach is an interesting alternative to avoid floating point arithmetic and its associated errors, it suffers from two facts: the integer weights distribution may be very large, and the alias table may lead to a smaller number of buckets without aliases. This can impact both the efficiency of the sampling step and the overall performance of the algorithm in large-scale simulations.

There is some literature to obtain a table with the maximum of triplets having their second objects empty. In this context the object generation could be done slightly faster. However, the decision problem of having constructed such a table is NP-complete. The reader will find important research directions in [14] and references therein.

## 3 Construction of the Alias table

Let us recall some notations. Let $X = [x_0, x_1, \dots x_{n-1}]$ of sequence of $n$ distinct objects and $W = [w_0, \dots, w_{n-1}]$ a sequence of non-negative integers such that object $x_i$ is of weight $w_i$. We denote the total weight by $w = \sum_i w_i$.

We aim at generating at random an object $x_i$ according to the global distribution $W$ avoiding any bias: the probability for generating $x_i$ is given by $w_i/w$, for each $i$.

For the rest of the paper, we suppose that the weights are positive integers, otherwise we remove all objects $x_i$ of weight 0. Furthermore we suppose that the greatest common divisor of integers in $W$ is $\texttt{GCD}(W) = 1$. If it is not the case, we replace $W$ by the normalized sequence $[\frac{w_0}{\texttt{GCD}(W)}, \dots, \frac{w_{n-1}}{\texttt{GCD}(W)}]$, still being an integer distribution. Obviously both sets of weights (the original one and the latter one) yield the same probability distribution. So, from now we suppose $W$ contains only positive integers whose GCD is 1.

**Definition 1.** *We define the cell size, denoted by* $\mathtt{cs}$*, to be the integer* $\lfloor \frac{w}{n} \rfloor$.

*Remark.* Since the distribution $W$ contains $n$ positive integers, we have $\mathtt{cs} \geq 1$.

We compute the Euclidean division of $w$ by $\mathtt{cs}$ and thus define the quotient $q$ and the remaining part $r$ such that

$$w = q \cdot \mathtt{cs} + r \qquad \text{with} \quad 0 \leq r < \mathtt{cs}. \tag{1}$$

Let us focus on the simple running example where $X$ contains $n = 5$ objects from $x_0$ to $x_4$ with respective weights $W = [2, 9, 1, 6, 1]$. The total weight is $w = 19$ and the cell size is $\mathtt{cs} = 3$. Thus we calculate $q = 6$ and $r = 1$.
From now, generally, we are faced in two cases:

- if $r > 0$, then we define a new object $x_n$, called *virtual object* and relate it to the weight $w_n = \mathtt{cs} - r$. Furthermore, we define $\bar{X} = [x_0, x_1, \dots x_{n-1}, x_n]$, $\bar{W} = [w_0, \dots, w_{n-1}, w_n]$ and $\bar{w} = w + w_n$.
- If $r = 0$, the we choose $\bar{X} = X$ and $\bar{W} = W$, and we set $x_n = \emptyset$.

Thus, we will use the following notations

$$\bar{w} = \bar{q} \cdot \mathtt{cs} \qquad \text{with either } \bar{q} = q + 1 \text{ or } \bar{q} = q. \tag{2}$$

*Remarks.* We observe now that $\bar{w}$ is a multiple of $\mathtt{cs}$. The value $\mathtt{cs}$ does not change, even if $\bar{X} \neq X$.

In our running example, we append a virtual object $x_5$ with weight 2 and then $\bar{q} = 7$.

Next algorithm builds a table $T$ with $\bar{q}$ cells, each one containing a triplet $(v', x', x)$ where $v'$ is an integer in $[\![1, \mathtt{cs}]\!]$[3] and $x$ and $x'$ being two distinct objects from $\bar{X}$. The object $x$ may eventually be $\emptyset$ in the case where $v' = \mathtt{cs}$, in that case we will say that $x'$ has no alias. As in the classical Alias method, the preprocessing step objective is to arrange the weight distribution. Here the weights $\bar{W}$ must be distributed into $\bar{q}$ cells, with the constraint that each cell contains at most two different objects whose associated weights cumulate to $\mathtt{cs}$.

Intuitively, we apply Vose's preprocessing step with specific inputs that are different from the original ones. Let us present our algorithm approach for this preprocessing step, building the alias table $T$. The algorithm proceeds in two phases. First objects are distributed in two stacks $L$ and $S$, containing respectively the larger objects (whose individual weights are larger than $\mathtt{cs}$) and the smaller objects of weights smaller than or equal to $\mathtt{cs}$. Then (1) the first step of the algorithm is to empty the stack $L$. To that order, each object of $S$ is completed with a part of a larger object from $L$ if necessary. In the case where the smaller object is of weight $\mathtt{cs}$ it is put alone, without alias, in its cell. Once done, either $S$ is also empty and the algorithm terminates (correctly) or from there (2) we will prove that $S$ will contain objects whose remaining weight are exactly $\mathtt{cs}$.

[3] The notation $[\![a, b]\!]$ stands for the subset of all consecutive integers from $a$ to $b$.

Hence each object is put without alias in a new cell of $T$. This approach will be proved correct.

Taking our running example, we get the following result for $T$:

$$[(2, x_5, x_3), (1, x_4, x_3), (3, x_3, \emptyset), (1, x_2, x_1), (2, x_0, x_1), (3, x_1, \emptyset), (3, x_1, \emptyset)].$$

**Algorithm 2** Building of the table of triplets $T$

**Input:** $\bar{X}$, $\bar{W}$, $\mathtt{cs}, \bar{q}$ previously defined.
**Output:** The table of triplets $T$.

```
1  function TableBuilding(X̄, W̄, cs, q̄)
2      T = init(q̄)
3      L = {x ∈ X̄ | W̄[x] > cs}                    ▷ L is the stack of the larger objects
4      S = {x ∈ X̄ | W̄[x] ≤ cs}                    ▷ S is the stack of the smaller objects
5      k = 0
6      while L is not empty do
7          x = pop(L)
8          if S is not empty then
9              x' = pop(S)
10             if W̄[x'] < cs then
11                 T[k] = (W̄[x'], x', x)
12                 W̄[x] = W̄[x] − (cs − W̄[x'])
13             else
14                 T[k] = (cs, x', ∅)
15         else
16             T[k] = (cs, x, ∅)
17             W̄[x] = W̄[x] − cs
18         if 0 < W̄[x] ≤ cs then
19             push(S, x)
20         else if W̄[x] > cs then
21             push(L, x)
22         k = k + 1
23     for x' ∈ S do
24         T[k] = (cs, x', ∅)          ▷ the objects still in S have a remaining weight equal to cs
25         k = k + 1
26     return T
```

*Remark.* The conditional cases in Lines 10 to 14 could be omitted by keeping only Line 12 with $W[x'] = \mathtt{cs}$; thus the alias object $x$ would have a weight 0 in this cell. The algorithm would still be correct.

**Proposition 3.** *The size $\bar{q}$ of the table $T$ satisfies $n \leq \bar{q} \leq (1 + \frac{1}{cs}) \cdot n - 1$.*

*Proof.* First suppose $q = \bar{q}$ and $w = \bar{w}$, i.e. $r = 0$ in Equation (2). In that case, using the latter equation, and the definition of $\mathtt{cs}$ we have

$$\frac{w}{q} = \lfloor \frac{w}{n} \rfloor.$$

Thus we deduce $q \geq n$. Furthermore, suppose now that $q \geq (1 + \frac{1}{\mathtt{cs}}) \cdot n$.

$$\mathtt{cs} = \lfloor \frac{q \cdot \mathtt{cs}}{n} \rfloor \geq \lfloor \frac{(1 + \frac{1}{\mathtt{cs}}) \cdot n \cdot \mathtt{cs}}{n} \rfloor \geq \mathtt{cs} + 1.$$

This is a contradiction. So in this case $n \leq q < (1 + \frac{1}{\mathtt{cs}}) \cdot n$.
Second, suppose in the Equation (1), we are in the case $r > 0$. From Equation (2) we obtain

$$\frac{w + (\mathtt{cs} - r)}{q+1} = \lfloor \frac{w}{n} \rfloor.$$

Thus we get

$$\frac{w}{q+1} \leq \lfloor \frac{w}{n} \rfloor \leq \frac{w}{n},$$

so $q + 1 \geq n$. Again, suppose $q \geq (1 + \frac{1}{\mathtt{cs}}) \cdot n - 1$. We deduce

$$\mathtt{cs} = \lfloor \frac{(q+1) \cdot \mathtt{cs}}{n} \rfloor \geq \lfloor \frac{(1 + \frac{1}{\mathtt{cs}}) \cdot n \cdot \mathtt{cs}}{n} \rfloor \geq \lfloor (1 + \frac{1}{\mathtt{cs}}) \mathtt{cs} \rfloor = \mathtt{cs} + 1.$$

This is a contradiction so we obtain in this second setting $n-1 \leq q < (1+\frac{1}{\mathtt{cs}}) \cdot n - 1$. Using the relationship between $q$ and $\bar{q}$ from (2), we get the proposition's statement. □

Note that these bounds for $\bar{q}$ are tight. In fact, if all the weights are equal (thus we are faced to the uniform distribution) once normalized each weight is 1, then $q = \bar{q} = n$. For the upper bound, fix two positive integers $\omega$ and $n$, and take a set of $\omega \cdot n$ objects, such that all of them except one have weight $\omega$ and the last one has weight $\omega \cdot n$. We calculate that the constant $\mathtt{cs}$ is equal to $\omega$, thus we get $q = (1 + \frac{1}{\mathtt{cs}}) \cdot \omega \cdot n - 1$ (recall $\omega \cdot n$ is the number of objects).

**Lemma 1.** *After each loop, starting Line 6 of Algorithm 2, the average updated weight of the objects still lying either in $L$ or in $S$ is greater than or equal to $\mathtt{cs}$.*

*Proof.* We prove the result by induction.

- At the beginning of the algorithm, let us prove that the average weight of the objects is larger than or equal to $\mathtt{cs}$.
  - In the case when $r = 0$ in Equation (1), we have $\bar{w} = w$ and the number of objects is $n$, thus the average weight is $\bar{w}/n = w/n \geq \mathtt{cs}$.
  - In the case when $r > 0$, we have $\bar{w} = w + (\mathtt{cs} - r) = (q+1)\mathtt{cs}$ and the number of objects is $n + 1$, thus the average weight satisfies

$$\frac{w}{n+1} < \frac{\bar{w}}{n+1} = \frac{(q+1)\mathtt{cs}}{n+1} < \frac{q+1}{n+1} \cdot \frac{w}{n} = \frac{q+1}{n} \cdot \frac{w}{n+1}.$$

    Proposition 3 induces $n \leq q + 1$, using the later inequality the case $n = q + 1$ is not possible, thus

$$\frac{\bar{w}}{n+1} = \frac{(q+1)\mathtt{cs}}{n+1} \geq \frac{(n+1)\mathtt{cs}}{n+1} = \mathtt{cs}.$$

  Hence when we enter the loop Line 6, the average weight of the objects distributed in $L$ and $S$ is greater than or equal to $\mathtt{cs}$.

– Suppose at the beginning of the loop, it remains $k$ objects distributed in $L$ and $S$, whose average weight $\omega \geq \texttt{cs}$.
During the loop, the total weight is decreased by $\texttt{cs}$.
There are two mutually exclusive loop-behaviors:
  - An object from $S$ is removed from $S$;
  - An object from $L$ is of updated weight greater than $\texttt{cs}$, its new weight is decreased by $\texttt{cs}$ and then, either it still stands in $L$ or it is pushed in $S$.

  In the first case, at the end of the loop, the cumulative remaining weight is $W' = k \cdot \omega - \texttt{cs}$, for $k-1$ objects (since $\omega$ was the average weight of the $k$ objects). If the number of remaining objects $k-1$ is positive, then the average weight is

$$\omega' = \frac{W'}{k-1} = \frac{k \cdot \omega - \texttt{cs}}{k-1} \geq \frac{k \cdot \texttt{cs} - \texttt{cs}}{k-1} = \texttt{cs}.$$

  Otherwise, when $k - 1 = 0$, the algorithm terminates.

  In the second case, the following constraints are valid : $S$ is empty and there is an element in $L$ of weight greater than $\texttt{cs}$. Thus the total weight $\bar{W}$ of objects remaining in $L$ at the beginning of the loop is at least $k \cdot (\texttt{cs} + 1) = k\texttt{cs} + k$. But the remaining weight is a multiple of $\texttt{cs}$, thus $\bar{W} \geq (k+1) \cdot \texttt{cs}$, since the number $k$ of objects does not decrease, so we deduce the updated average weight

$$\frac{W'}{k} = \frac{\bar{W} - \texttt{cs}}{k} \geq \frac{(k+1) \cdot \texttt{cs} - \texttt{cs}}{k} = \texttt{cs}.$$

  So in both cases, at the end of the loop, the average weight of the remaining objects is at least $\texttt{cs}$.

Thus the lemma is proved. □

**Lemma 2.** *At Line 27 of Algorithm 2, all objects still lying in $S$ have a weight equal to $\texttt{cs}$.*

*Proof.* Arriving at Line 27, the loop invariant proved in Lemma 1 is still valid, thus the average weight of the objects still lying in $S$ is greater than or equal to $\texttt{cs}$. Hence we deduce that these remaining objects have all a weight equal to $\texttt{cs}$. □

**Theorem 1.** *Algorithm 2 is correct.*

The correctness of the approach lies essentially in proving that the algorithm terminates, $T$ being completed.

*Proof.* This is a direct corollary of Lemmas 1 and 2. Each cell of $T$ contains a cumulated weight of $\texttt{cs}$, either using a single object or by completing an object with an alias. □

## 4 Uniform random generation and its analysis

In the case where we have a virtual object in the alias table $T$, the approach puts it as the third component in the last cell of $T$. This behavior is dictated by our stack implementation.

Let us now introduce our sampling algorithm with virtual object rejection.

**Algorithm 3** Random sampling with virtual object rejection

```
Input: T, cs, and virtual_object.
Output: A random object from T \ {virtual_object}.
1  function RandomGenerationOld(T, cs, virtual_object)
2      n = len(T)
3      result = virtual_object
4      while result == virtual_object do
5          i = Uniform(n)
6          b = Bernoulli(T[i][0], cs)
7          if b then
8              result = T[i][1]
9          else
10             result = T[i][2]
11     return result
```

Optimal algorithms, for the number of random bits that are needed, are implemented in our repository. Their efficiency has been proved by Lumbroso [6]. This optimality concerns the number of random bits, which is not the same criterion as running time; in Section 5, we thus also evaluate a second implementation of our sampler that favors time over bits. Even if both calls to Uniform and Bernoulli are optimal, the overall algorithm is not optimal in the random bits consumption.

**Proposition 4.** *The consumption of random bits is in average upper bounded by*

$$\left(1 + \frac{\mathtt{cs} - r}{w}\right)(\log(n) + 3) \leq \left(1 + \frac{1}{n}\right)(\log(n) + 3)\,.$$

We say that the sampled object is rejected $t$ times when the **While** loop (Line 4 in Algorithm 3) is executed $t + 1$ times.

**Lemma 3.** *The probability that the sampled object is rejected $t$ times is*

$$\mathbb{P}(\textit{exactly } t \textit{ rejections}) = \left(\frac{\mathtt{cs} - r}{\bar{w}}\right)^t \left(\frac{w}{\bar{w}}\right).$$

*The average number of rejections is equal to* $\mathbb{E}(\textit{number of rejections}) = \dfrac{\mathtt{cs} - r}{w}$.

*Proof.* The case when the sampled object is rejected has probability $\frac{\mathtt{cs}-r}{\bar{w}}$ that corresponds to the ratio of the weight of the virtual object and the total weight. Thus having exactly $t$ rejections has probability $\left(\frac{\mathtt{cs}-r}{\bar{w}}\right)^t$ and then the next object

is valid (is not the virtual object) with probability $\left(\frac{w}{\bar{w}}\right)$. Thus the first statement is proved. To derive the average number of rejections, we have to calculate

$$\mathbb{E}(\text{number of rejections}) = \sum_{t=0}^{\infty} t \left(\frac{\mathtt{cs} - r}{\bar{w}}\right)^t \left(\frac{w}{\bar{w}}\right).$$

The next step is classical and can be derived from the derivative of the geometric series $\sum_{t=1}^{\infty} t x^{t-1} = \frac{1}{(1-x)^2}$ for $|x| < 1$. Hence,

$$\mathbb{E}(\text{number of rejections}) = \frac{w}{\bar{w}} \cdot \frac{\mathtt{cs} - r}{\bar{w}} \cdot \sum_{t=1}^{\infty} t \left(\frac{\mathtt{cs} - r}{\bar{w}}\right)^{t-1} = \frac{\mathtt{cs} - r}{w}.$$

And the proof is complete. □

The proof of Proposition 4 is a direct consequence of Lemma 3.

## 5 Experiments and reasons for using alias method for integers

This section is built on our code repository, which is available at `GITHUB`. All the competitive methods are implemented in C, using arbitrary precision arithmetic and a flow of cryptographic random bits. A part of the implementation is devoted to reproduce our experimental results. Experiments were conducted on a MacBook Pro (Apple M4 Pro chip, 14-core CPU) with 48GB of unified memory.

### 5.1 Overview of the experimental approach

This section evaluates the practical performance of the algorithms introduced in Sections 3 and 4 with a focus on measurable scalability limits. Our evaluation targets five quantities: preprocessing time, sampling throughput, memory footprint, cryptographic random-bit consumption and savings. First, all computations use exact integer arithmetic, so results are free from floating-point rounding artifacts. Second, we use highly compact data structures, which makes explicit the trade-off at the core of our approach: entropic efficiency (measured in bits) versus time efficiency.

A distinctive feature of our implementations is support for arbitrary precision arithmetic. To the best of our knowledge, this is the only approach in our comparison that natively provides exact sampling under arbitrary precision. Random sampling is coupled with a dedicated background thread that continuously refills a bitstream. This design enables the use of cryptographic random bits, which are expensive to obtain in practice, with a limited impact on performance, because bit production is decoupled from sampling and overlapped in parallel (the reported times include the CPU time of this thread). All these features enable scaling in several independent directions: the number of objects can grow without a fixed-width bound, and the weight of each object can also grow arbitrarily large.

This property is essential for high-precision and large-distribution regimes where fixed-size numeric types become a bottleneck.

Beyond of our new algorithm, we implemented four variants of algorithm provided in the literature: FLDR, ALDR, Rust-Alias, and Fraction-Alias Samplers, all in arbitrary precision arithmetic. While these implementations provide exactness and numerical robustness, none of them scales satisfactorily across all target regimes. The remainder of this section quantifies these limits through a comparative analysis of memory footprint, preprocessing cost, random-bit consumption, and sampling throughput.

### 5.2 Case Study: Combinatorial Chemistry

What is following illustrates how exact sampling becomes essential when the underlying combinatorial space is too large, as in several nowadays applications. The chemical space, i.e. the set of all combinatorially plausible stable molecules, is far too vast to be enumerated. Reference databases report sizes ranging from $9.7 \cdot 10^7$ molecules for the smallest structures (e.g., GDB-17, up to 17 heavy atoms) to several trillion compounds for modern make-on-demand combinatorial libraries [15], and beyond roughly thirty heavy atoms the number of possible structures far exceeds $2^{64} \approx 1.8 \cdot 10^{19}$. Consequently, any statistical analysis based on a subset of this space is structurally biased unless the sample is representative of the true underlying distribution.

To address this context, Monterrubio-Chanca and von Rudorff [7] proposed a method for producing unbiased, representative samples of chemical space without ever enumerating it, applicable to molecules containing on the order of 30 heavy atoms. Obviously, this is a regime in which exhaustive enumeration is impossible. The method proceeds in five steps:

1. Generation of all theoretically possible chemical formulas by integer partitioning;
2. Grouping of these formulas by degree sequence;
3. Distinction between "pure" and "non-pure" sequences, depending on whether several elements of the same valence appear;
4. Estimation of the size of each group, either through exact counting or through a heuristic based on the graph's average path length when exact counting proves too costly;
5. Weighted random sampling among the groups, proportional to their estimated size, followed by the sampling of a representative molecule within the chosen group.

We note that the two steps **1.** and **5.** are classical in the context of combinatorics and combinatorial algorithm design. Especially step **5.** is a classical application of the Alias method, which is the focus of this paper. All their results are reproducible through their open-source `nablachem` Python package [8].

Using their library, we derive the distribution of chemical molecules containing 32 heavy atoms, among Carbon, Hydrogen, Oxygen, Nitrogen, and Fluorine. The

total number of molecules is $N = 20\,484$, and the sum of their weights is about $1.25 \cdot 2^{168} \sim 4.7 \cdot 10^{50}$. Thus in the following benchmarks for the sampling method, we used these parameters, which seem representative of the actual combinatorial chemistry research. Remark that step **5.** cannot be performed without bias in their library based on floating-point arithmetic samplers that are not precise enough in this regime. More details about this case study case be found in our code repository.

### 5.3 Algorithms with arbitrary precision integers

The goal of these experiments is to compare our approach with existing algorithms for discrete random sampling. Following the design of our Integer-Alias Sampler, the next phase focuses on an experimental evaluation of our algorithm against existing methods (adapted with arbitrary precision). We start from the code repository of Saad et al. [12] in order to add our implementation in their comparison: we thus are able to run all their benchmarks. We have adapted the original memory consumption analysis to the precise size used by the data structures built by the preprocessing step.

While the original implementations by Saad et al. compared several different literature approaches to emphasize random bit consumption, it is more relevant in our context to retain only the top three methods from the literature, according to our efficiency constraints (time, memory, and random bit consumption), while addressing our main challenge of safety at scale via arbitrary-precision arithmetic. Consequently, we compare the following algorithmic approaches:

- FLDR Sampler: Saad et al. [12] that we adapted for arbitrary-precision integers;
- ALDR Sampler: Saad et al. [1], an improvement of FLDR in the context of random bit consumption, that we also adapted for arbitrary-precision integers;
- Rust-Alias Sampler: Rust Developers, a variant of Walker version coded in Rust with integer arithmetic [11], where we also have modified types to support arbitrary-precision integers;
- Fraction-Alias Sampler: our new exact version of Walker-Alias using exact rational fractions (with arbitrary precision) instead of floating-point numbers;
- our Integer-Alias Sampler: our new algorithmic approach, in two implementations sharing the same table. *v1* optimizes random-bit consumption, with Lumbroso's bit-optimal UNIFORM and BERNOULLI [6]; *v2* optimizes running time, with Lemire's multiply-and-shift UNIFORM [5], the Bernoulli test reusing the same 64-bit random word, at the cost of 64 random bits per draw.

We deliberately leave floating-point samplers (e.g., Walker–Vose with IEEE doubles) out of the comparison: they are approximate, and here the approximation is not marginal. In 179 of our 201 distributions, some objects have a share $Nw_i/W$ of their bucket below $2^{-53}$, the resolution of a double-precision threshold (17 525 of the 20 484 objects for the median distribution): a floating-point sampler would

never draw them. Its running time would thus be that of another distribution, and is no meaningful reference; this is precisely the regime our exact samplers are designed for.

The experimental protocol starts by fixing the distribution size (the number of objects), that we take to be $N = 20\,484$ and the total weight distribution to be the one computed in the context of combinatorial chemistry (see Section 5.2) $W \approx 4.7 \cdot 10^{50}$. The measurements were collected on a set of random distributions following a Dirichlet distribution. Using a Dirichlet distribution allows varying the entropy while maintaining fixed values for parameters $N$ and $W$. The distribution itself is not of major importance, once we can choose for the average weight of an object or the entropy of the resulting random distribution (e.g. a gaussian distribution of weights would have given the same kind of experimental simulations). The observed entropy corresponds to Shannon entropy. The latter is a central theoretical measure in the context of random sampling. We refer the reader to the seminal paper of Knuth and Yao [3] for a general overview. Essentially, we note that the closer a distribution is to a uniform distribution, the higher its entropy (bounded by $\log_2(N)$). Conversely, a strongly biased distribution (e.g., one element with a very large weight and many elements with a weight of 1) results in lower entropy. For each distribution, among the 200 random ones, 10 millions of objects have been sampled to measure the average sampling time and the average number of random bits consumed.

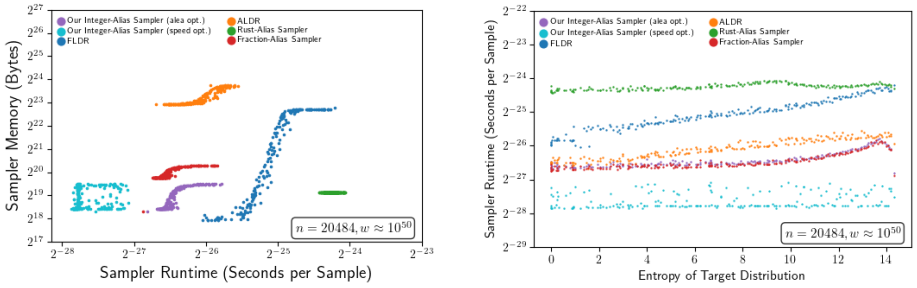



Fig. 1: Sampling runtime of the compared algorithms under arbitrary precision, against the memory footprint (left) and the entropy of the distributions (right)

Figure 1 (left) plots the runtime of the sampling step against the memory footprint of the internal data structures, with each dot corresponding to one random distribution (averaged over 10 million samples). Remark that $2^{-20}\text{s} = 0.95\,\mu\text{s}$ and $2^{20}\text{B} = 1\text{MB}$. Beyond individual algorithm comparisons, the spread of each cluster is informative: the tight horizontal grouping of Alias-based methods reflects a fundamental algorithmic property, namely that their $O(1)$ sampling step is structurally independent of the target distribution, so every input distribution yields essentially the same runtime. By contrast, FLDR and ALDR exhibit a much wider horizontal spread, exposing their sensitivity to the shape of the distribution.

The resulting picture traces an implicit Pareto frontier in the (runtime, memory) plane: FLDR and ALDR occupy the fast-but-memory-intensive region for favorable distributions, while Alias-based approaches occupy the compact-and-predictable region. Within the Alias family, our Integer-Alias Sampler consistently shifts the cluster to the left of Rust-Alias at comparable memory cost, demonstrating that integer normalization by `cs` rather than by the full cumulative weight $W$ produces a concrete speedup in the sampling step, independently of the distribution. Its v2 implementation is the fastest of all compared samplers, about 2.3 times faster than v1 (median over the distributions).

Let us now make the entropy explicit as an independent variable to understand its role in the runtime, memory footprint, and preprocessing time.

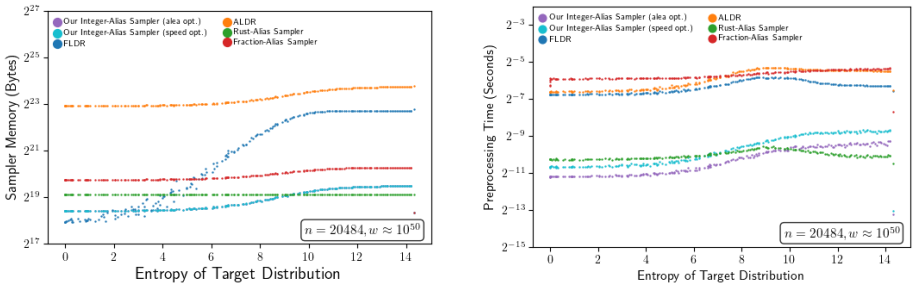



Fig. 2: Memory footprint (left) and preprocessing time (right) of the compared algorithms under arbitrary precision, as a function of the entropy of the distributions

Figure 1 (right) confirms that Alias-based runtimes are near-constant across the full entropy spectrum. This invariance is not merely an empirical regularity: it is a direct consequence of the preprocessing step encoding the entire distribution into the alias table, so that each draw requires only a uniform bucket selection and a single Bernoulli test, regardless of the distributional complexity. The entropy-dependence of FLDR and ALDR has a different, more fundamental cause: both algorithms are built on the DDG-tree framework of Knuth and Yao [3], in which random bits are consumed one at a time to navigate a binary decision tree. The expected number of bits per draw is approximately equal to the Shannon entropy of the distribution, and since each bit operation carries a fixed cost, the average runtime scales accordingly. As a result, FLDR and ALDR are *faster* at low entropy (skewed distributions produce short, predictable tree paths) and *slower* as entropy increases toward $\log_2(N)$ (near-uniform distributions require paths of maximal depth). The crossover at roughly half the maximum entropy (beyond which our Integer-Alias Sampler outperforms ALDR in runtime) reflects the point where the entropy-proportional cost of ALDR exceeds the constant cost of the Alias approach. The combinatorial chemistry application of Section 5.2, with its strongly skewed weight distribution falls in the low-entropy regime where

FLDR and ALDR are at their most competitive; yet even there, our Integer-Alias Sampler remains within the same order of magnitude in runtime while providing strictly superior memory efficiency.

Regarding memory (Figure 2, left), Alias-based approaches are more compact than ALDR across all entropies, and than FLDR except at low entropy, where its tree is small. Note that we report the exact memory footprint of the data structure produced by the preprocessing step. Within the Alias family, two observations are worth noting. First, even though the integer weights used by the Rust-Alias Sampler are $N$ times larger in value than those of our Integer-Alias Sampler (since Rust takes the total weight $W$ as the target cell capacity rather than `cs`) the resulting table requires at most one additional cell, so both approaches yield similar memory footprints. Ours nevertheless grows with the entropy: our table stores each threshold with its significant bytes only, and the thresholds of light objects get longer as the distribution flattens. This confirms that the runtime advantage of Integer-Alias observed in Figure 1 (left) stems from the arithmetic cost of the Bernoulli step on large integers, not from a cache or layout effect. Second, the Fraction-Alias Sampler's table is approximately twice as large as those of the other Alias approaches: this is a direct consequence of storing exact rational numbers (numerator and denominator in arbitrary precision) rather than a single integer per entry, and it quantifies the memory cost of the more general rational representation.

Figure 2 (right) reports the preprocessing time, i.e., the construction of the data structure from the weights, file parsing excluded (for FLDR, whose construction is written in Python in [12], we use a C port producing the same encoding). Our Integer-Alias Sampler, together with Rust-Alias, has the cheapest preprocessing: below 2.5 ms on every distribution, about twenty times less than FLDR, ALDR and the Fraction-Alias Sampler (10 to 25 ms). It only performs a linear number of integer operations on the weights, whereas FLDR and ALDR scan the binary expansion of every weight at each level of their DDG tree, and Fraction-Alias normalizes a rational number after each operation.

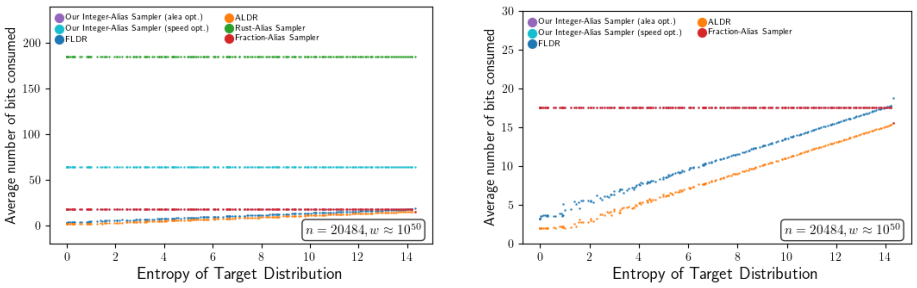



Fig. 3: Randomness consumption of the compared algorithms under arbitrary precision, as a function of the entropy of the distributions. The right plot zooms into the left one, capping the vertical axis at 30 bits

Figure 3 reports the average number of random bits consumed per sample. The rightmost plot zooms into the leftmost one by capping the vertical axis at 30 bits, excluding Rust-Alias, whose consumption is an order of magnitude higher, and our v2 implementation (64 bits per draw). This anomaly has a structural cause: instead of sampling a Bernoulli variable, in the Rust approach requires drawing a uniform integer in $[0, W)$ where $\log_2(W) \approx 153$ bits (far beyond the $\log_2(N) \approx 14$ bits needed by our Integer-Alias Sampler). This is the direct price of the absence of weight normalization in the Rust implementation, and it renders that approach impractical for large-precision regimes regardless of its time efficiency.

Among the remaining four algorithms, FLDR and ALDR consume fewer random bits than our Integer-Alias Sampler across all entropies, with FLDR achieving the information-theoretic optimum of Knuth and Yao [3]. However, this entropic advantage narrows as entropy increases. In the high-entropy regime (distributions close to uniform), the dominant cost for any sampler is the uniform bucket selection, which requires $\log_2(N)$ bits regardless of the algorithm. As the distribution approaches uniform, this fixed cost dominates, and the overhead of our Integer-Alias Sampler relative to the optimum becomes negligible. In this limit, our algorithm thus simultaneously achieves $O(1)$ sampling time, minimal memory footprint, and near-optimal bit consumption (realizing, in the regime most favorable to all competing objectives, the best combination of all three efficiency criteria).

**Acknowledgments.** This work was supported by the ANR-FWF project PANDAG ANR-23-CE48-0014.

**Disclosure of Interests.** The authors have no competing interests to declare that are relevant to the content of this article.

## References

1. Draper, T., Saad, F.: Efficient rejection sampling in the entropy-optimal range. IEEE Transactions on Information Theory **72**(5), 2801–2822 (2026)
2. Genitrini, A., Pépin, M.: Lexicographic Unranking of Combinations Revisited. Algorithms **14**(3) (2021)
3. Knuth, D., Yao, A.: The complexity of nonuniform random number generation. In: Traub, J.F. (ed.) Algorithms and Complexity: New Directions and Recent Results. pp. 357–428. Academic Press, New York (1976)
4. Kreher, D.L., Stinson, D.R.: Combinatorial Algorithms: generation, enumeration, and search. CRC Press (1999)
5. Lemire, D.: Fast random integer generation in an interval. ACM Transactions on Modeling and Computer Simulation **29**(1), 3:1–3:12 (2019)
6. Lumbroso, J.: Optimal discrete uniform generation from coin flips, and applications. CoRR **abs/1304.1916** (2013)
7. Monterrubio-Chanca, D., von Rudorff, G.: Representative random sampling of chemical space. J. of Chemical Theory and Computation **21**(24), 12810–12819 (2025)

8. Nablachem Lab: nablachem: Python package for representative sampling of chemical space (2025), https://archive.softwareheritage.org/swh:1:dir:18fd766273c869ddb9012d1e5aca6e65aa377351
9. Nijenhuis, A., Wilf, H.S.: Combinatorial algorithms. Computer science and applied mathematics, Academic Press, New York, NY (1975)
10. Ruskey, F.: Combinatorial Generation (2003)
11. Rust Programming Language: Weighted_rand (2025), https://archive.softwareheritage.org/swh:1:dir:41dec5677c27f049668b4fc380183d89db18ca75/
12. Saad, F., Freer, C., Rinard, M., Mansinghka, V.: Optimal approximate sampling from discrete probability distributions. Proc. ACM Program. Lang. **4**(POPL) (2019)
13. Skiena, S.: The Algorithm Design Manual, Third Edition. Texts in Computer Science, Springer (2020)
14. Smith, J., Jacobson, S.: An analysis of the alias method for discrete random-variate generation. INFORMS J. on Computing **17**(3), 321–327 (2005)
15. Su, J., Song, F., Huang, D., Liao, M.: Constructing and characterizing trillion-scale combinatorial chemical library. J. of Cheminformatics **18**(1) (2026)
16. Vose, M.: A linear algorithm for generating random numbers with a given distribution. IEEE Transactions on Software Engineering **17**(9), 972–975 (1991)
17. Walker, A.: New fast method for generating discrete random numbers with arbitrary frequency distributions. Electronics Letters **10**(8), 127–128 (1974)
18. Walker, A.: An Efficient Method for Generating Discrete Random Variables with General Distributions. ACM Trans. Math. Softw. **3**(3), 253–256 (1977)